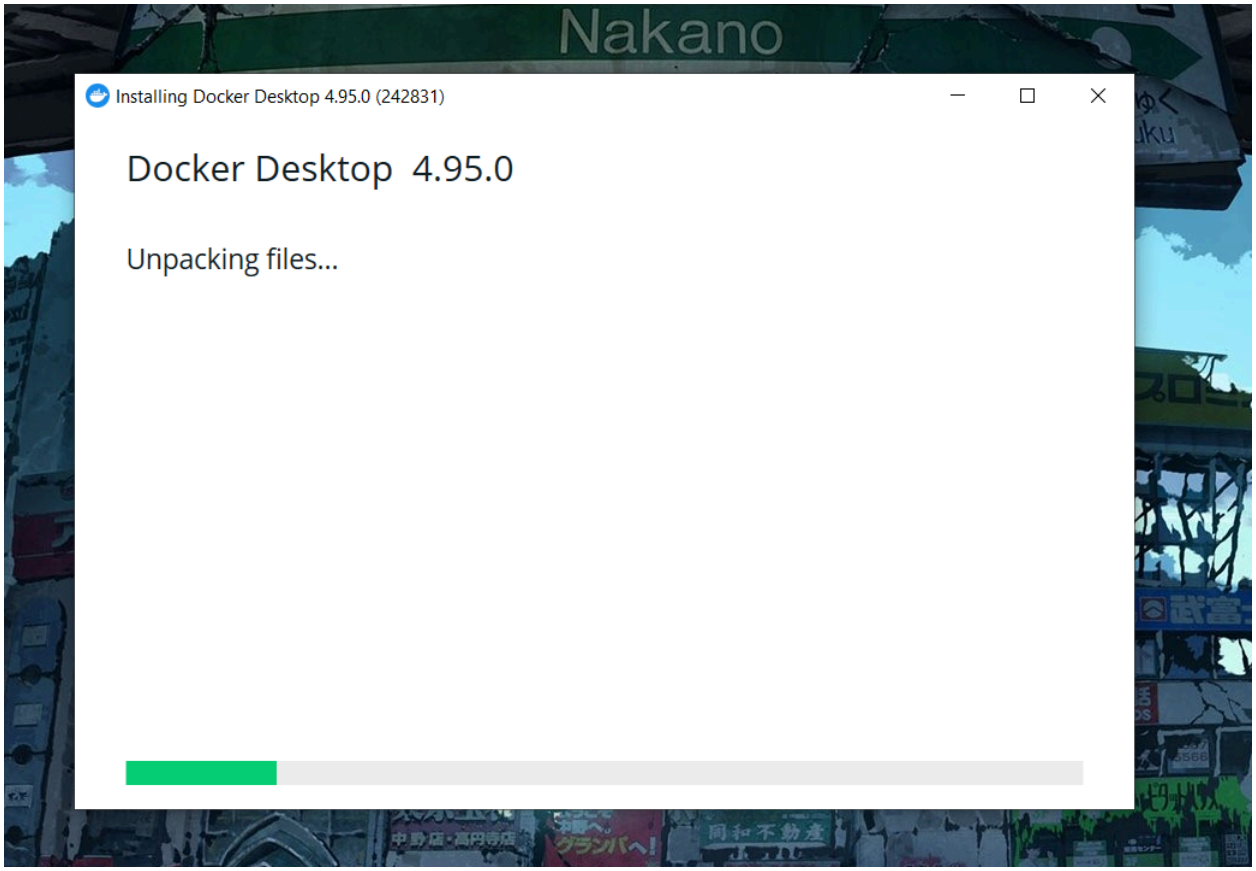
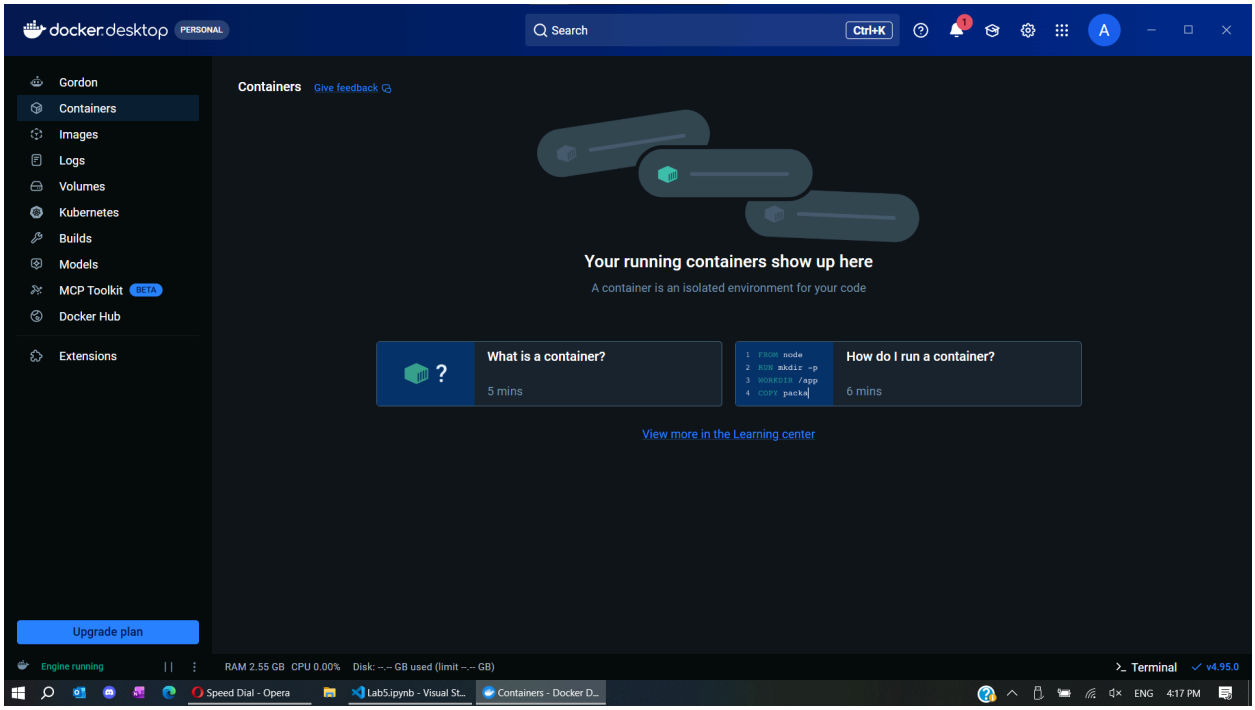
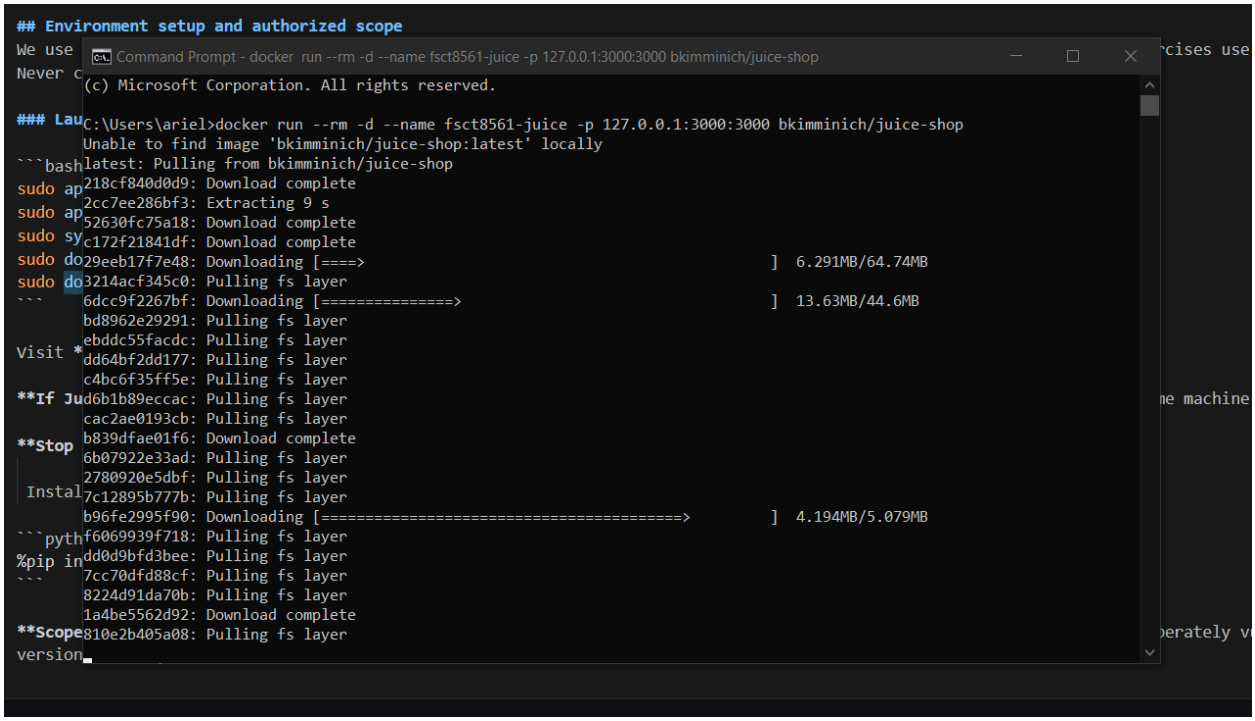
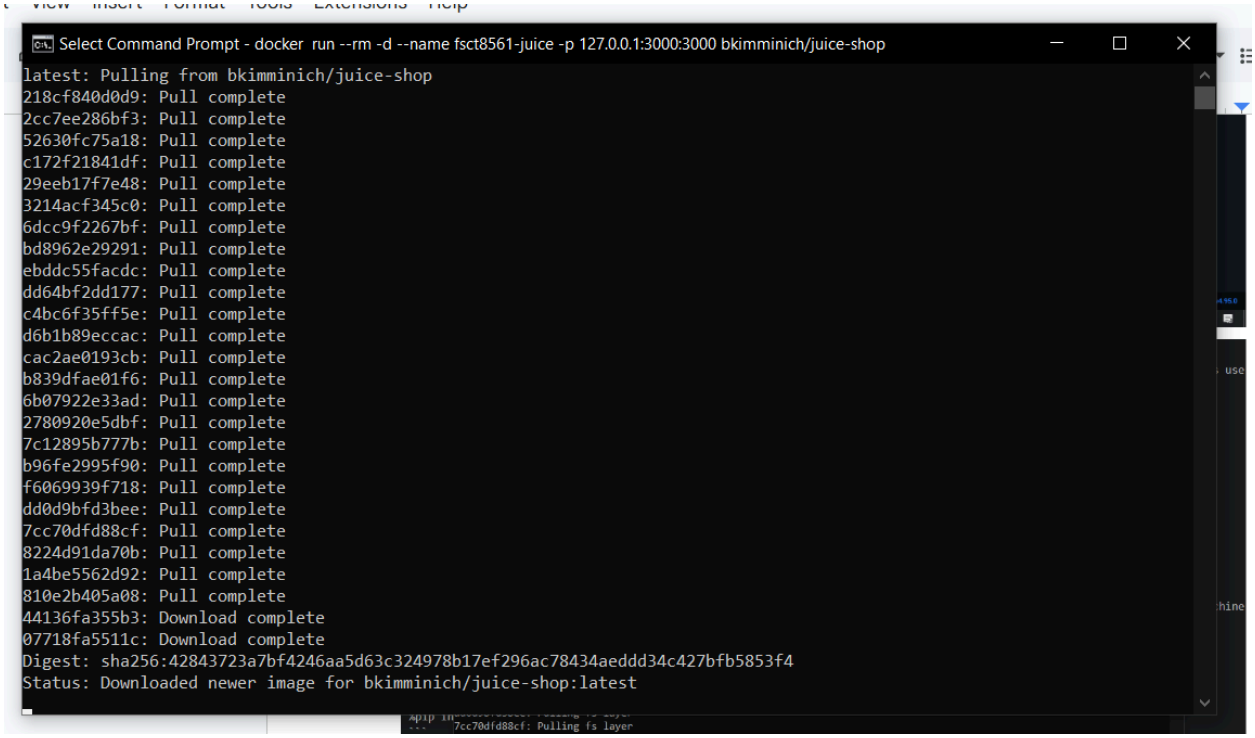
Lab 5: Automated Web Vulnerability Analysis Using Python

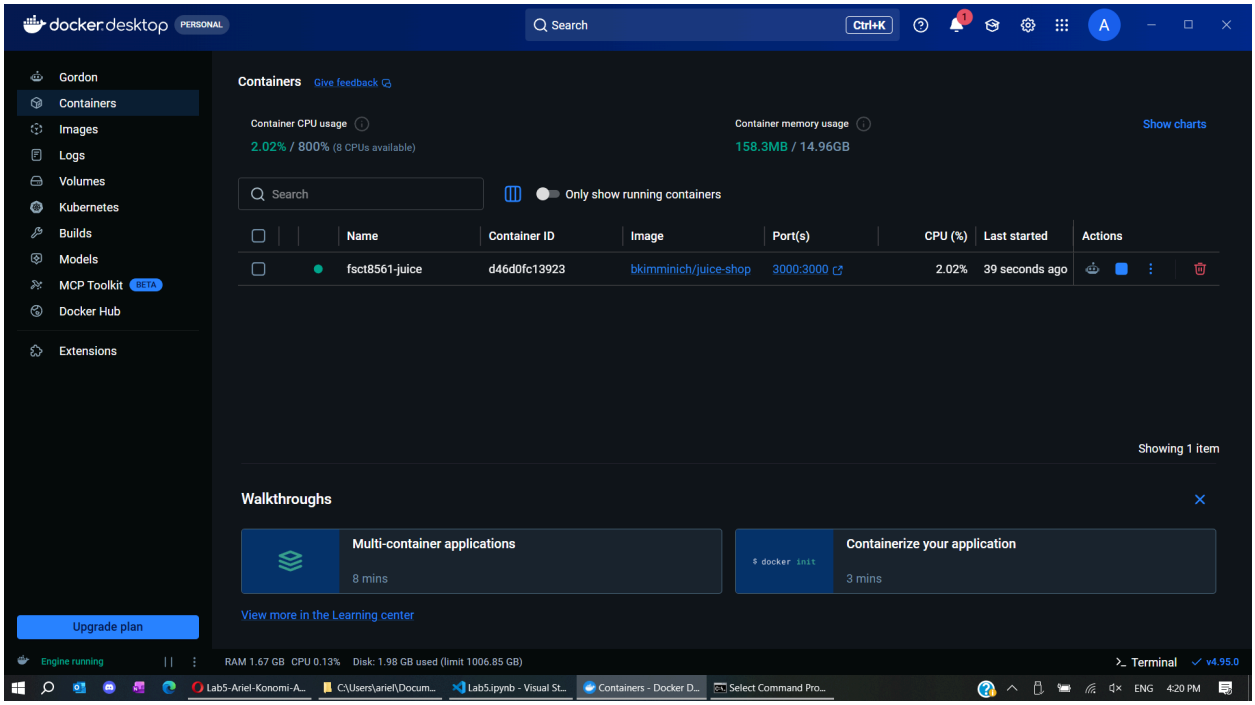
Environment setup

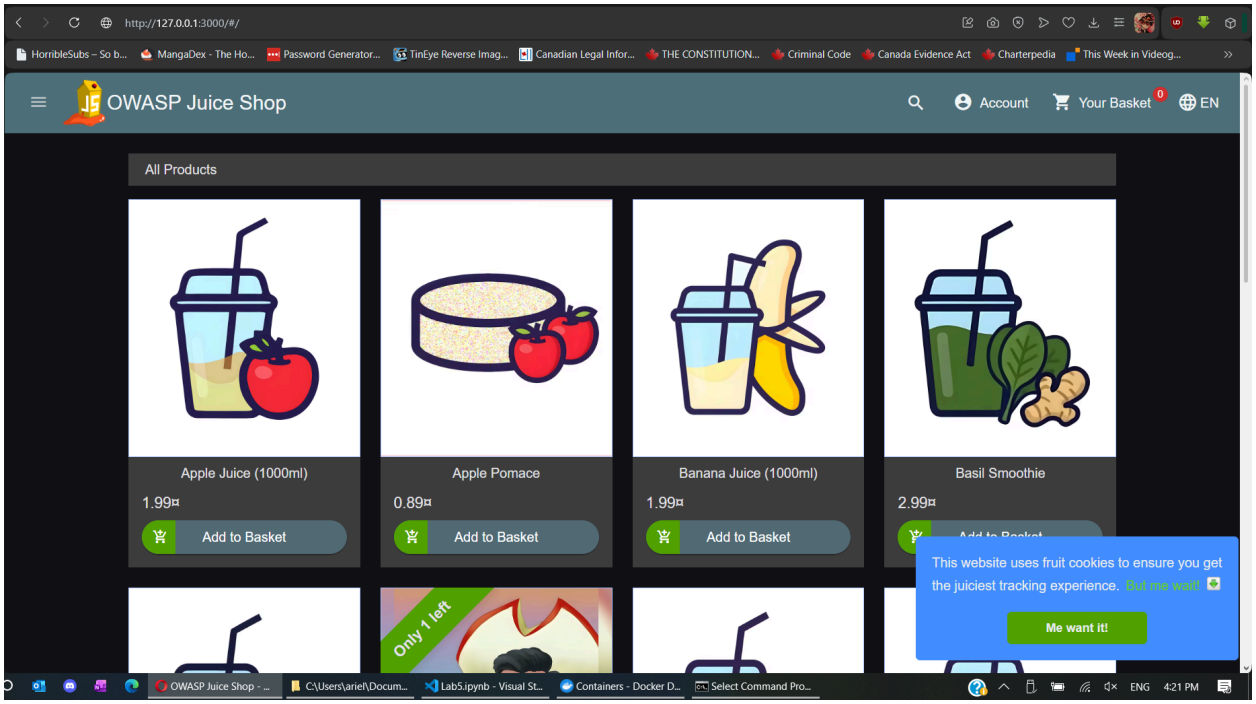



docker desktop
PERSONAL
Search
Ctrl+K
Gordon
Containers
Images
Logs
Volumes
Kubernetes
Builds
Models
MCP Toolkit
BETA
Docker Hub
Extensions
Upgrade plan
Containers
Give feedback
Your running containers show up here
A container is an isolated environment for your code
What is a container?
5 mins
How do I run a container?
6 mins
View more in the Learning center
Engine running
RAM 2.55 GB CPU 0.00% Disk: --.-- GB used (limit --.-- GB)
Terminal
v4.95.0
Speed Dial - Opera
Lab5.ipynb - Visual St...
Containers - Docker D...
ENG
4:17 PM


Environment setup and authorized scope
Command Prompt - docker run --rm -d --name fsct8561-juice -p 127.0.0.1:3000:3000 bkimminich/juice-shop
(c) Microsoft Corporation. All rights reserved.
C:\Users\ariel>docker run --rm -d --name fsct8561-juice -p 127.0.0.1:3000:3000 bkimminich/juice-shop
Unable to find image 'bkimminich/juice-shop:latest' locally
latest: Pulling from bkimminich/juice-shop
218cf840d0d9: Download complete
2cc7ee286bf3: Extracting 9 s
52630fc75a18: Download complete
c172f21841df: Download complete
29eeb17f7e48: Downloading [====>] 6.291MB/64.74MB
3214acf345c0: Pulling fs layer
6dcc9f2267bf: Downloading [==========>] 13.63MB/44.6MB
bd8962e29291: Pulling fs layer
ebddc55facdc: Pulling fs layer
dd64bf2dd177: Pulling fs layer
c4bc6f35ff5e: Pulling fs layer
d6b1b89eccac: Pulling fs layer
cac2ae0193cb: Pulling fs layer
b839dfae01f6: Download complete
6b07922e33ad: Pulling fs layer
2780920e5dbf: Pulling fs layer
7c12895b777b: Pulling fs layer
b96fe2995f90: Downloading [==========>] 4.194MB/5.079MB
f6069939f718: Pulling fs layer
dd0d9bfd3bee: Pulling fs layer
7cc70dfd88cf: Pulling fs layer
8224d91da70b: Pulling fs layer
1a4be5562d92: Download complete
810e2b405a08: Pulling fs layer

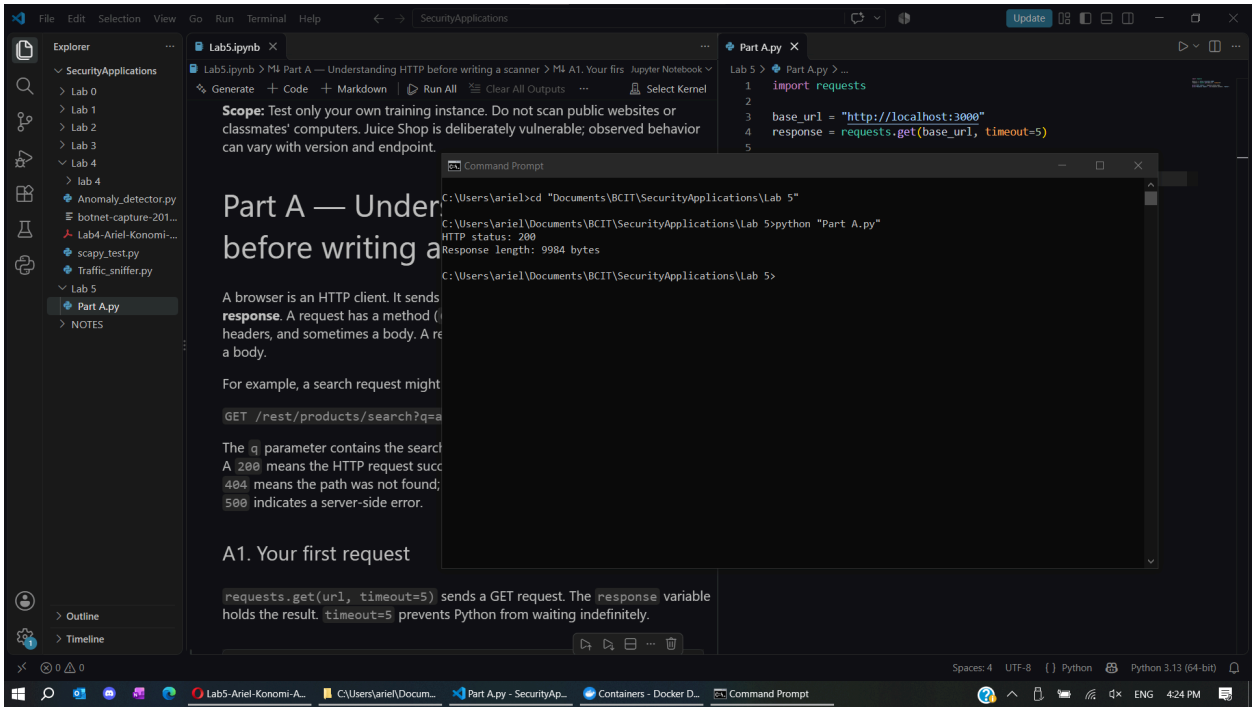

Select Command Prompt - docker run --rm -d --name fsct8561-juice -p 127.0.0.1:3000:3000 bkimminich/juice-shop
latest: Pulling from bkimminich/juice-shop
218cf840d0d9: Pull complete
2cc7ee286bf3: Pull complete
52630fc75a18: Pull complete
c172f21841df: Pull complete
29eeb17f7e48: Pull complete
3214acf345c0: Pull complete
6dcc9f2267bf: Pull complete
bd8962e29291: Pull complete
ebddc55facdc: Pull complete
dd64bf2dd177: Pull complete
c4bc6f35ff5e: Pull complete
d6b1b89eccac: Pull complete
cac2ae0193cb: Pull complete
b839dfae01f6: Pull complete
6b07922e33ad: Pull complete
2780920e5dbf: Pull complete
7c12895b777b: Pull complete
b96fe2995f90: Pull complete
f6069939f718: Pull complete
dd0d9bfd3bee: Pull complete
7cc70dfd88cf: Pull complete
8224d91da70b: Pull complete
1a4be5562d92: Pull complete
810e2b405a08: Pull complete
44136fa355b3: Download complete
07718fa5511c: Download complete
Digest: sha256:42843723a7bf4246aa5d63c324978b17ef296ac78434aeddd34c427bfb5853f4
Status: Downloaded newer image for bkimminich/juice-shop:latest


docker desktop
PERSONAL
Search
Ctrl+K
Gordon
Containers
Images
Logs
Volumes
Kubernetes
Builds
Models
MCP Toolkit
BETA
Docker Hub
Extensions
Upgrade plan
Containers
Give feedback
Container CPU usage
2.02% / 800% (8 CPUs available)
Container memory usage
158.3MB / 14.96GB
Show charts
Search
Only show running containers
Name
Container ID
Image
Port(s)
CPU (%)
Last started
Actions
fsct8561-juice
d46d0fc13923
bkimminich/juice-shop
3000:3000
2.02%
39 seconds ago
Showing 1 item
Walkthroughs
Multi-container applications
8 mins
Containerize your application
3 mins
View more in the Learning center
Engine running
RAM 1.67 GB CPU 0.13% Disk: 1.98 GB used (limit 1006.85 GB)
Terminal
v4.95.0
ENG
4:20 PM

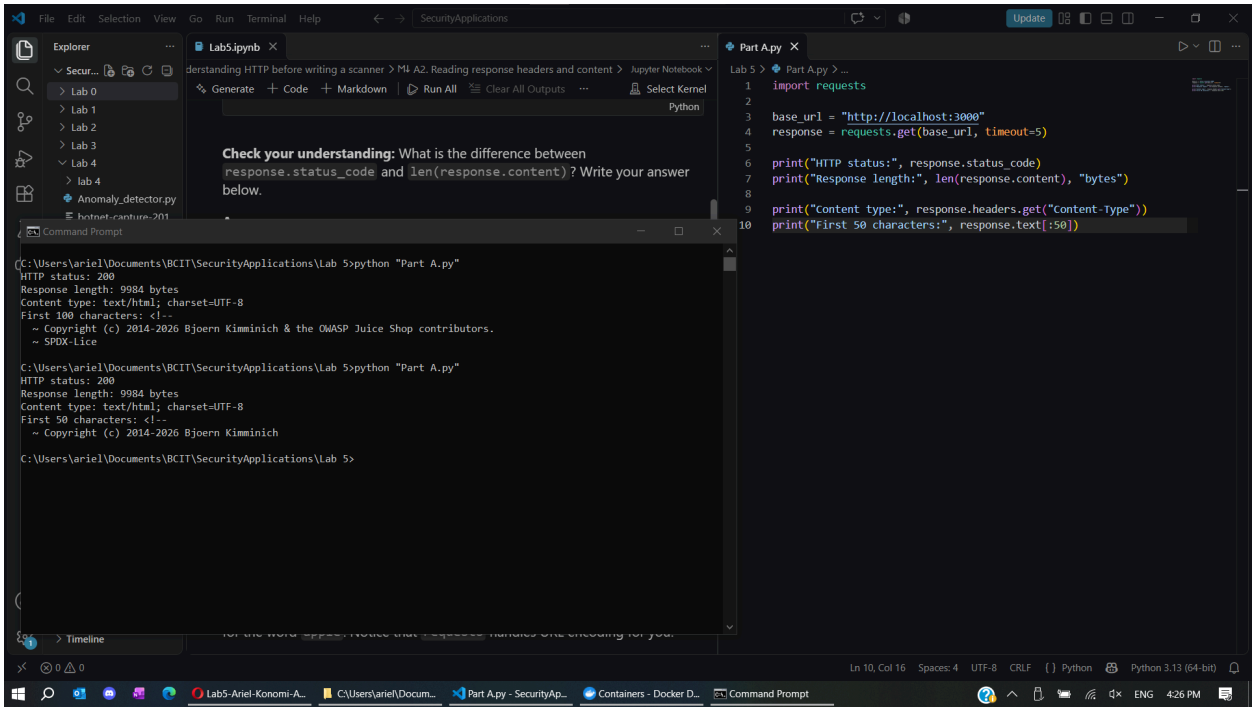

http://127.0.0.1:3000/#/
OWASP Juice Shop
Account
Your Basket
EN
All Products
Apple Juice (1000ml)
1.99¤
Add to Basket
Apple Pomace
0.89¤
Add to Basket
Banana Juice (1000ml)
1.99¤
Add to Basket
Basil Smoothie
2.99¤
Only 1 left
This website uses fruit cookies to ensure you get the juiciest tracking experience.
Me want it!

Part A

A1


Lab5.ipynb
Part A.py
Scope: Test only your own training instance. Do not scan public websites or classmates' computers. Juice Shop is deliberately vulnerable; observed behavior can vary with version and endpoint.
import requests
base_url = "http://localhost:3000"
response = requests.get(base_url, timeout=5)
Command Prompt
C:\Users\ariel>cd "Documents\BCIT\SecurityApplications\Lab 5"
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>python "Part A.py"
HTTP status: 200
Response length: 9984 bytes
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>
A1. Your first request

A2

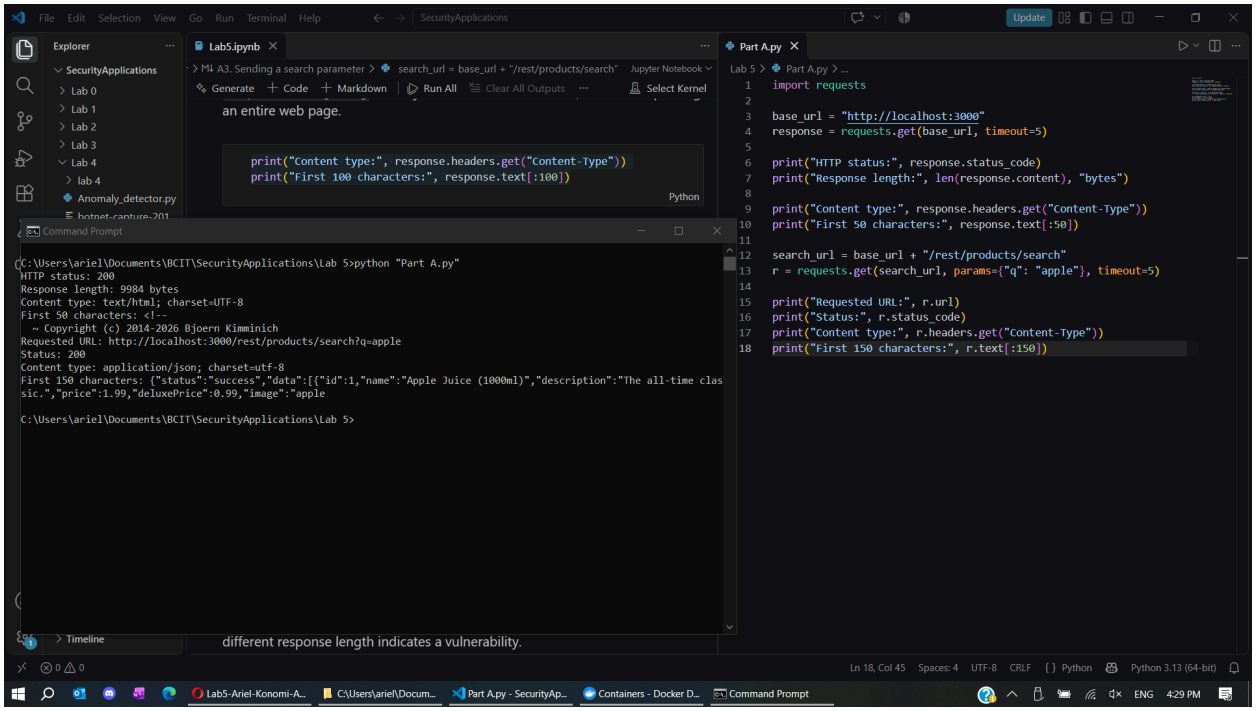


Try it:

Explain what slicing `[:50]` does

Slicing '[:50]' limits the output string to print only the first 50 characters of the response

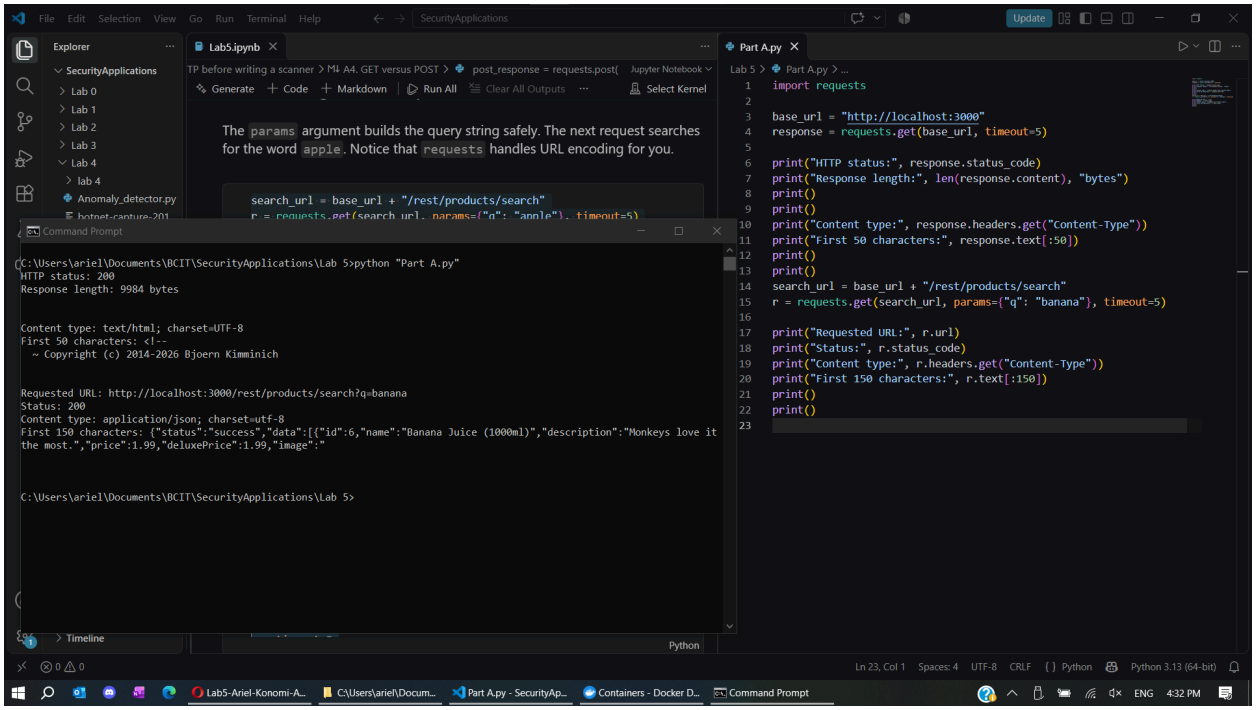
A3



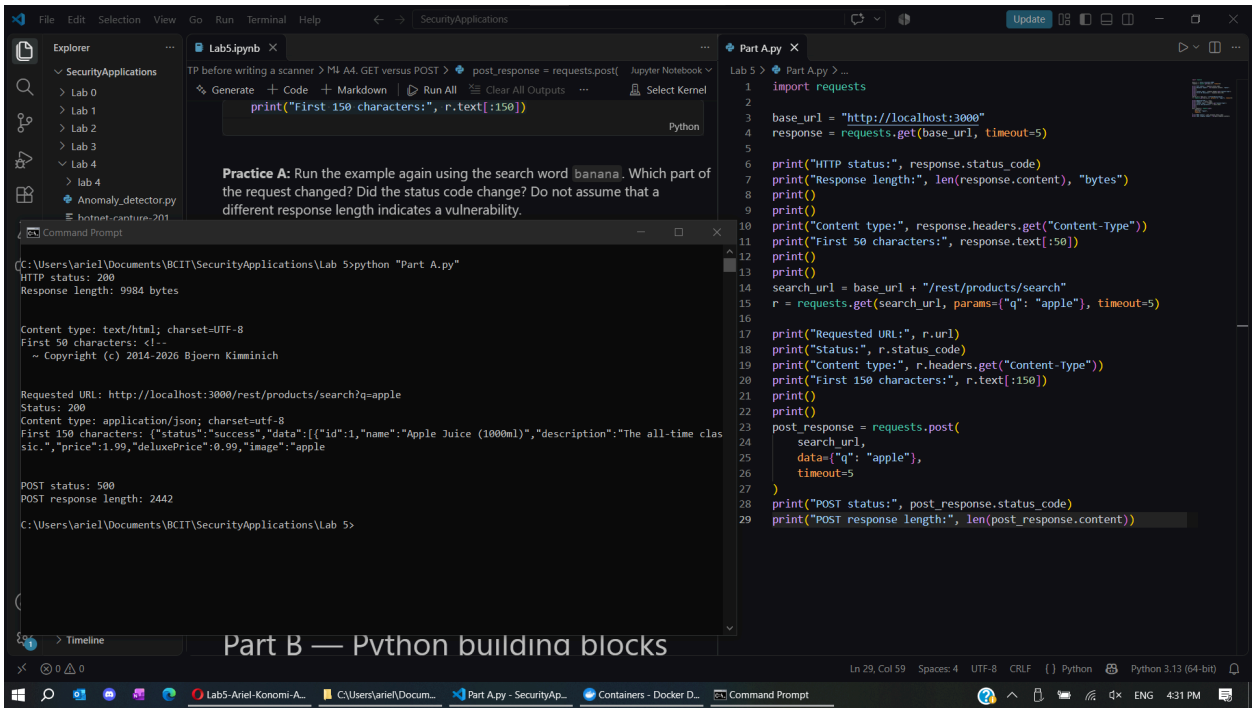
Practice A:

Run the example again using the search word `banana`. Which part of the request changed? Did the status code change? Do not assume that a different response length indicates a vulnerability.

The query parameter changed from q=apple to q=banana. The code in this case remains 200 as the products match. The response length might differ due to them being two different products



A4



Part B — Python building blocks

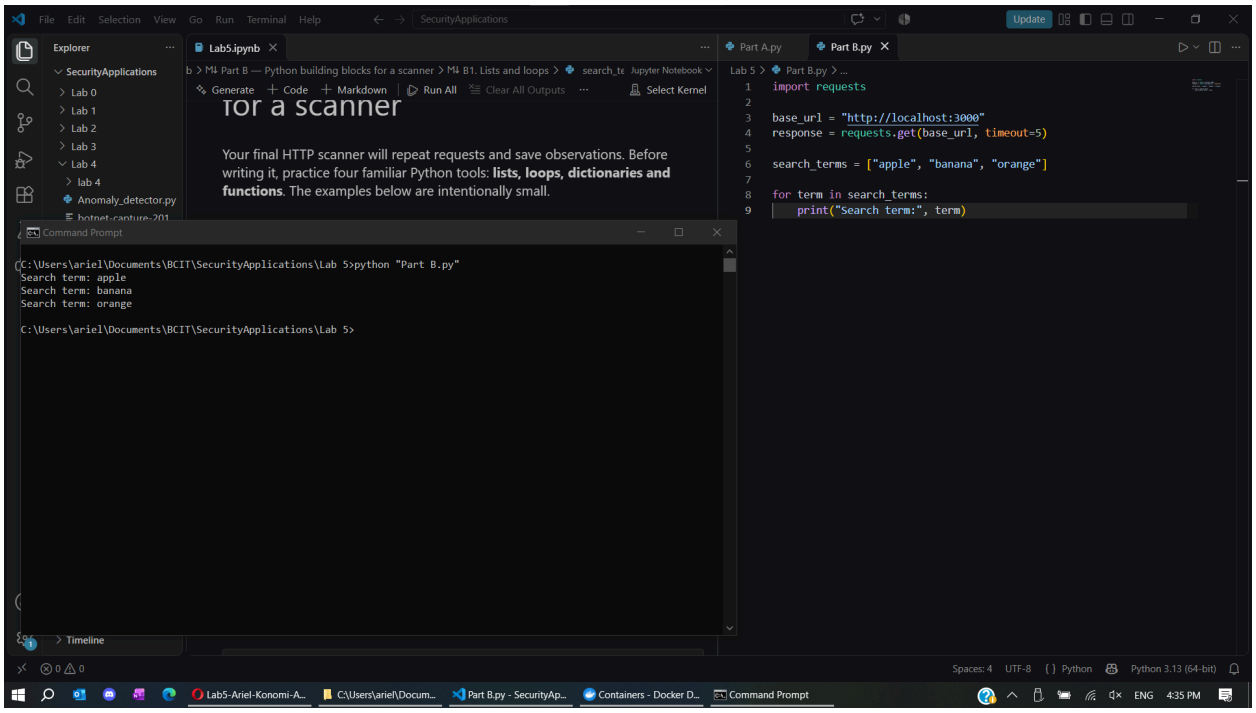
Practice B:

In one or two sentences, compare the GET and POST results. Why does a `405` response not necessarily indicate a vulnerability?

GET requests retrieve data via query parameters. POST requests submit data. A 405 response does not necessarily indicate a vulnerability as the endpoint might simply just not accept that HTTP method

Part B

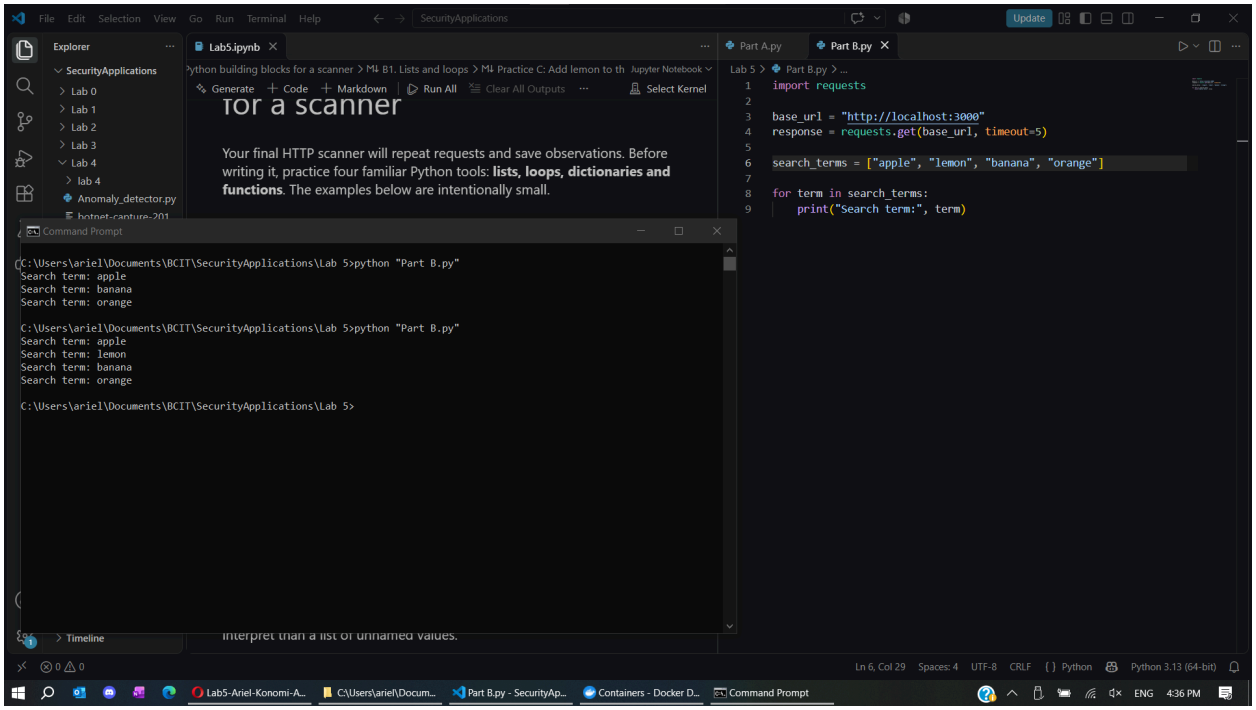
B1



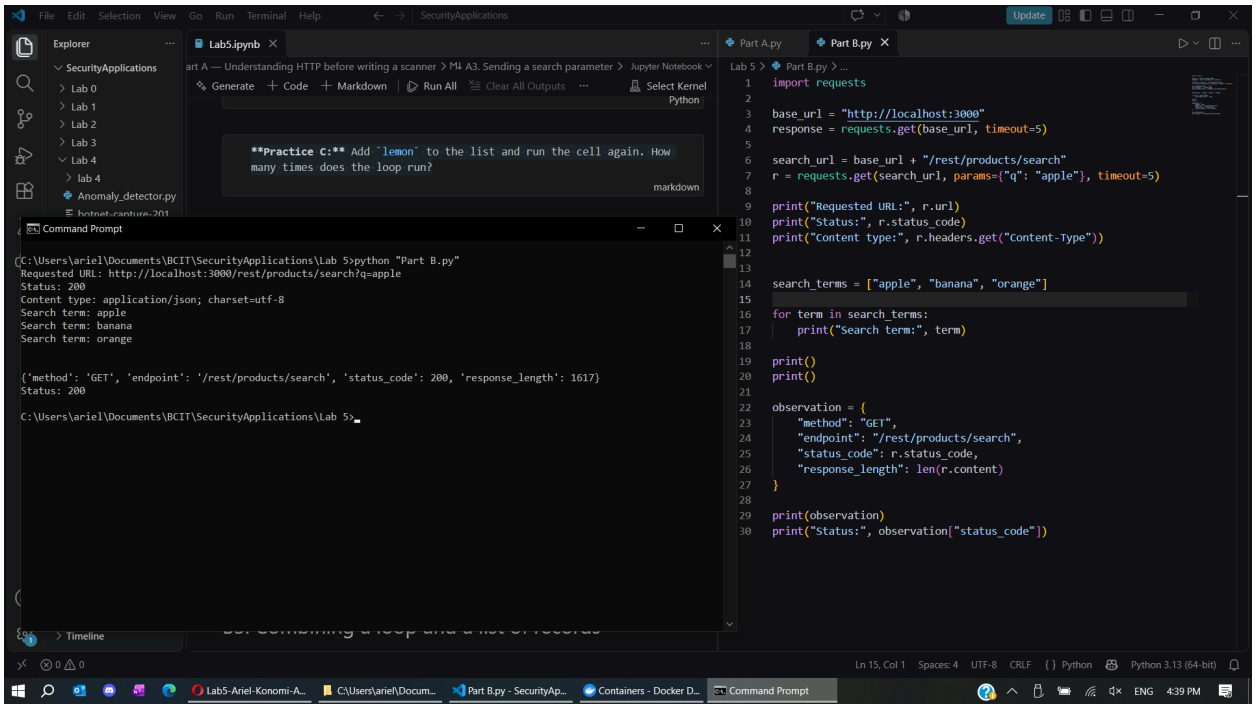
Practice C:

Add `lemon` to the list and run the cell again. How many times does the loop run?

The loop runs once more, for a total of four times


Part B.py
import requests
base_url = "http://localhost:3000"
response = requests.get(base_url, timeout=5)
search_terms = ["apple", "lemon", "banana", "orange"]
for term in search_terms:
print("Search term:", term)
Command Prompt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>python "Part B.py"
Search term: apple
Search term: banana
Search term: orange
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>python "Part B.py"
Search term: apple
Search term: lemon
Search term: banana
Search term: orange
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>

B2


Part B.py
import requests
base_url = "http://localhost:3000"
response = requests.get(base_url, timeout=5)
search_url = base_url + "/rest/products/search"
r = requests.get(search_url, params={"q": "apple"}, timeout=5)
print("Requested URL:", r.url)
print("Status:", r.status_code)
print("Content type:", r.headers.get("Content-Type"))
search_terms = ["apple", "banana", "orange"]
for term in search_terms:
print("Search term:", term)
print()
print()
observation = {
"method": "GET",
"endpoint": "/rest/products/search",
"status_code": r.status_code,
"response_length": len(r.content)
}
print(observation)
print("Status:", observation["status_code"])
Command Prompt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>python "Part B.py"
Requested URL: http://localhost:3000/rest/products/search?q=apple
Status: 200
Content type: application/json; charset=utf-8
Search term: apple
Search term: banana
Search term: orange
{'method': 'GET', 'endpoint': '/rest/products/search', 'status_code': 200, 'response_length': 1617}
Status: 200
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>

Practice D:

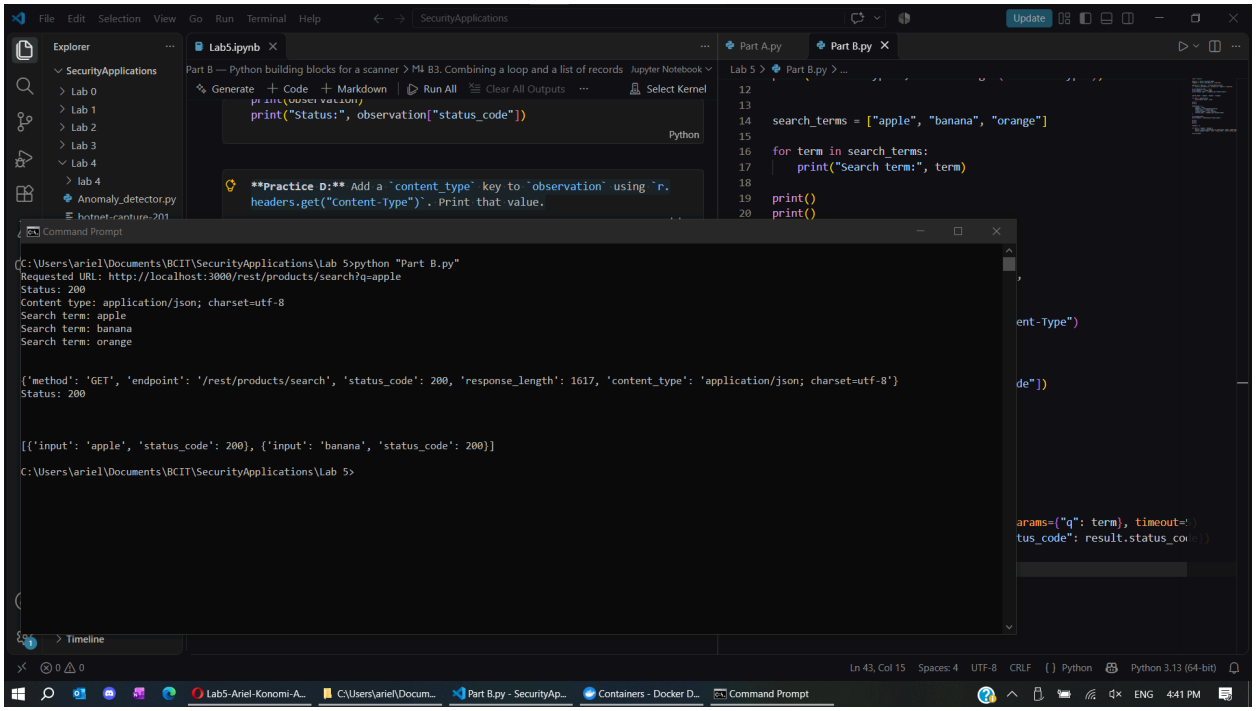
Add a `content_type` key to `observation` using `r.headers.get("Content-Type")`. Print that value.

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>python "Part B.py"
Requested URL: http://localhost:3000/rest/products/search?q=apple
Status: 200
Content type: application/json; charset=utf-8
Search term: apple
Search term: banana
Search term: orange


{'method': 'GET', 'endpoint': '/rest/products/search', 'status_code': 200, 'response_length': 1617, 'content_type': 'application/json; charset=utf-8'}
Status: 200

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>
```

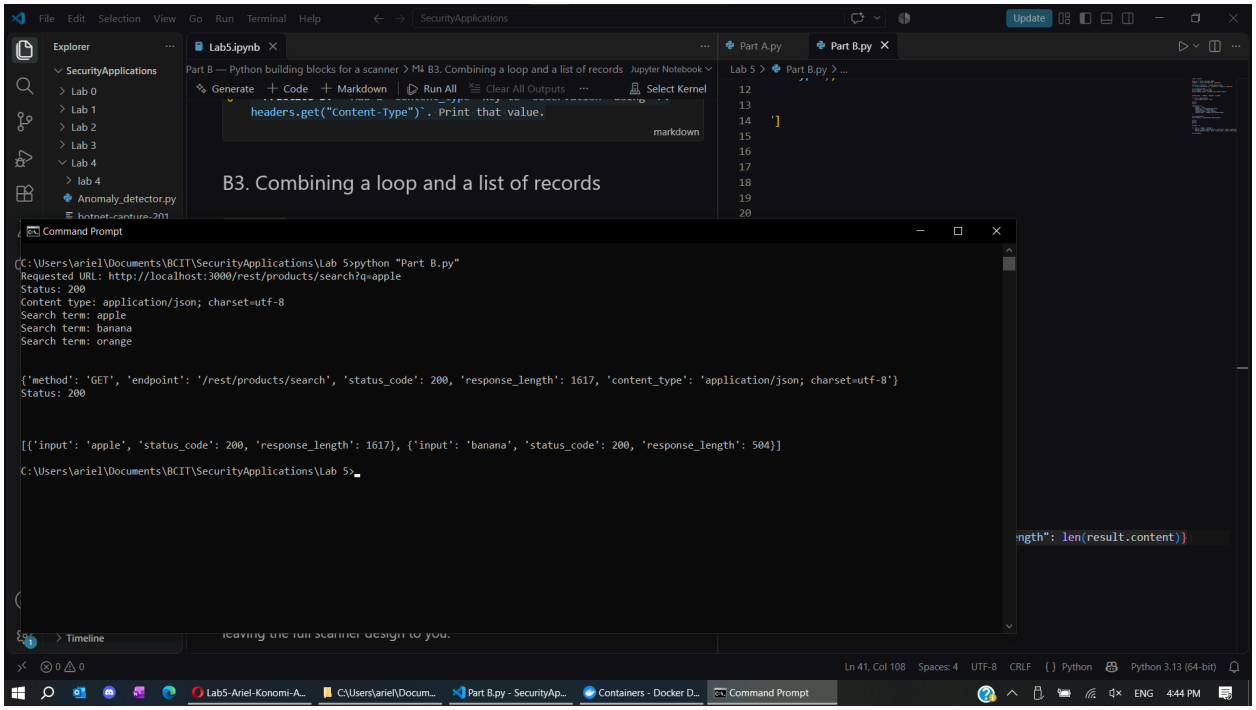
B3



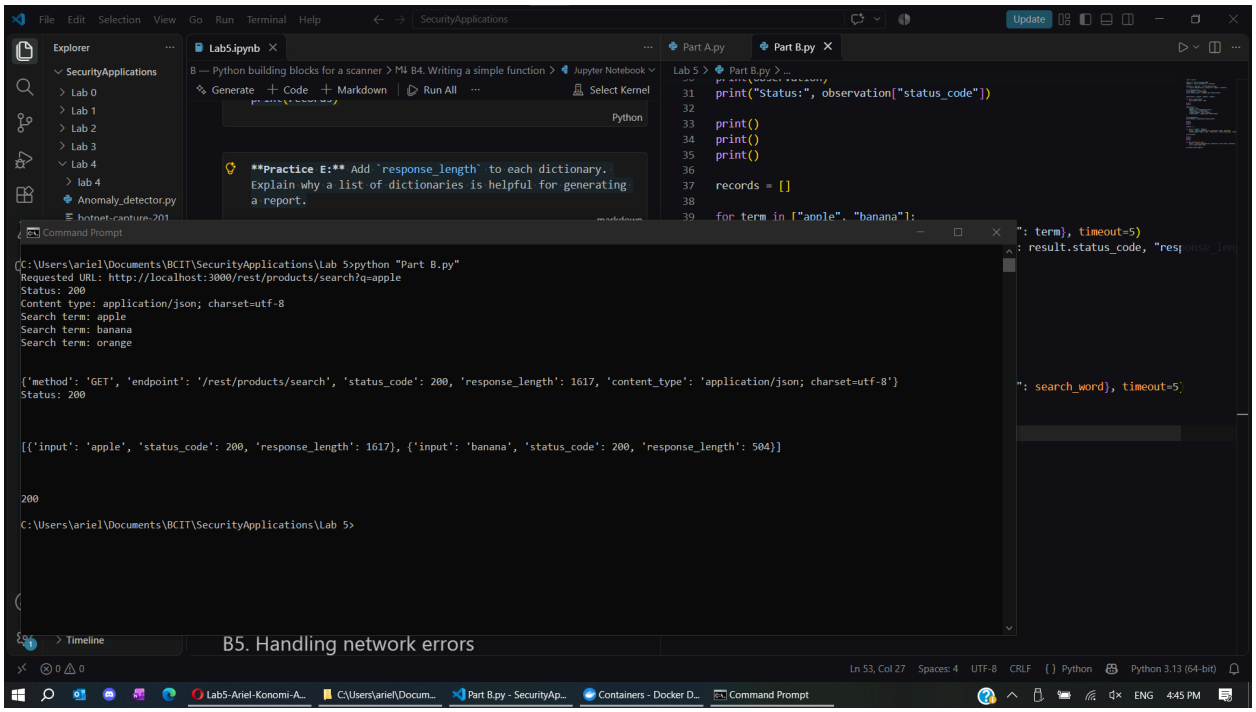
Practice E:

Add `response_length` to each dictionary. Explain why a list of dictionaries is helpful for generating a report.

A list of dictionaries is helpful for generating a report as it represents each request and stores the details for each. They can then be exported into a CSV file

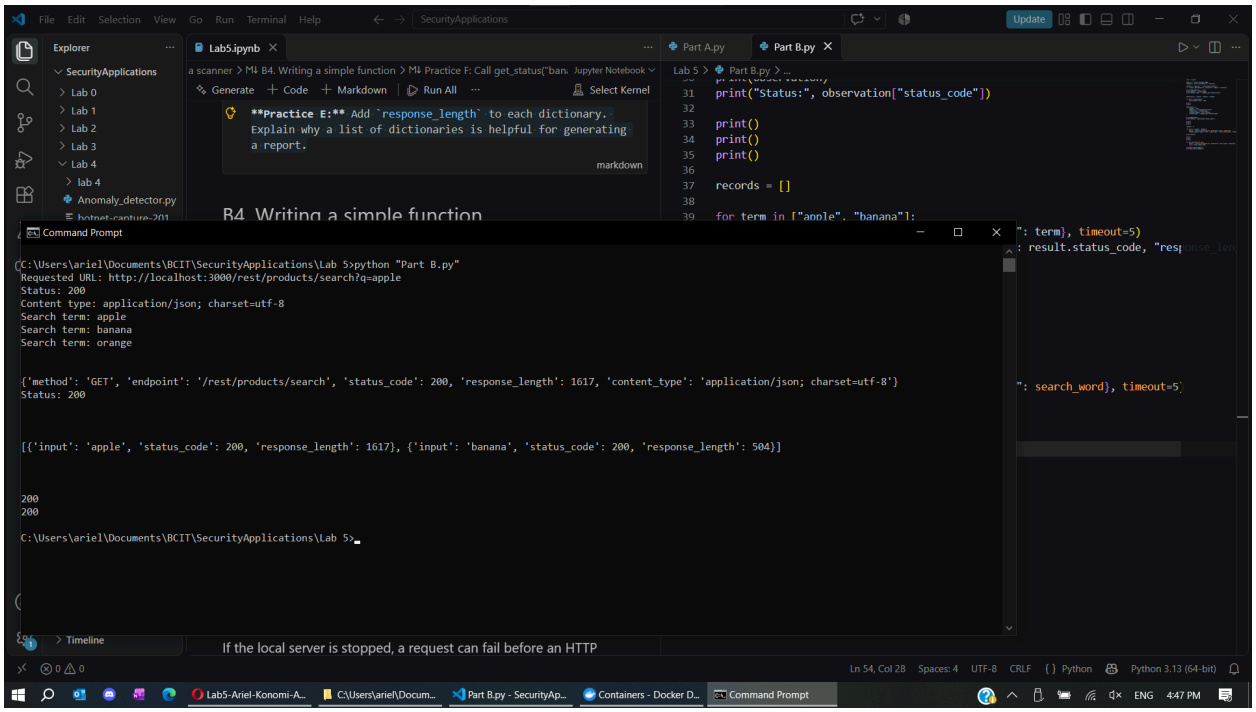

Command Prompt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>python "Part B.py"
Requested URL: http://localhost:3000/rest/products/search?q=apple
Status: 200
Content type: application/json; charset=utf-8
Search term: apple
Search term: banana
Search term: orange
{'method': 'GET', 'endpoint': '/rest/products/search', 'status_code': 200, 'response_length': 1617, 'content_type': 'application/json; charset=utf-8'}
Status: 200
[{'input': 'apple', 'status_code': 200, 'response_length': 1617}, {'input': 'banana', 'status_code': 200, 'response_length': 504}]
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>
B3. Combining a loop and a list of records

B4


Command Prompt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>python "Part B.py"
Requested URL: http://localhost:3000/rest/products/search?q=apple
Status: 200
Content type: application/json; charset=utf-8
Search term: apple
Search term: banana
Search term: orange
{'method': 'GET', 'endpoint': '/rest/products/search', 'status_code': 200, 'response_length': 1617, 'content_type': 'application/json; charset=utf-8'}
Status: 200
[{'input': 'apple', 'status_code': 200, 'response_length': 1617}, {'input': 'banana', 'status_code': 200, 'response_length': 504}]
200
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>
B5. Handling network errors

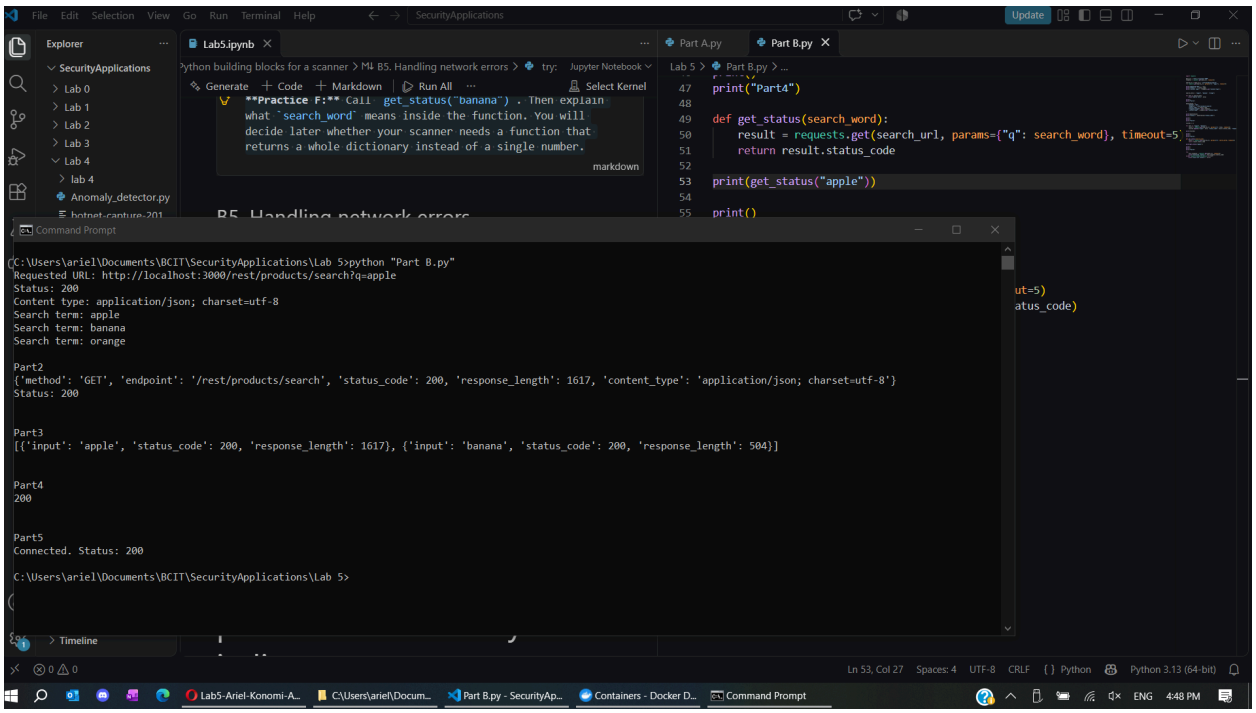
Practice F:

Call `get_status("banana")`. Then explain what `search_word` means inside the function. You will decide later whether your scanner needs a function that returns a whole dictionary instead of a single number.



Search_word is a parameter that receives the value passed on to the function. This function can be used with multiple and different search terms

B5



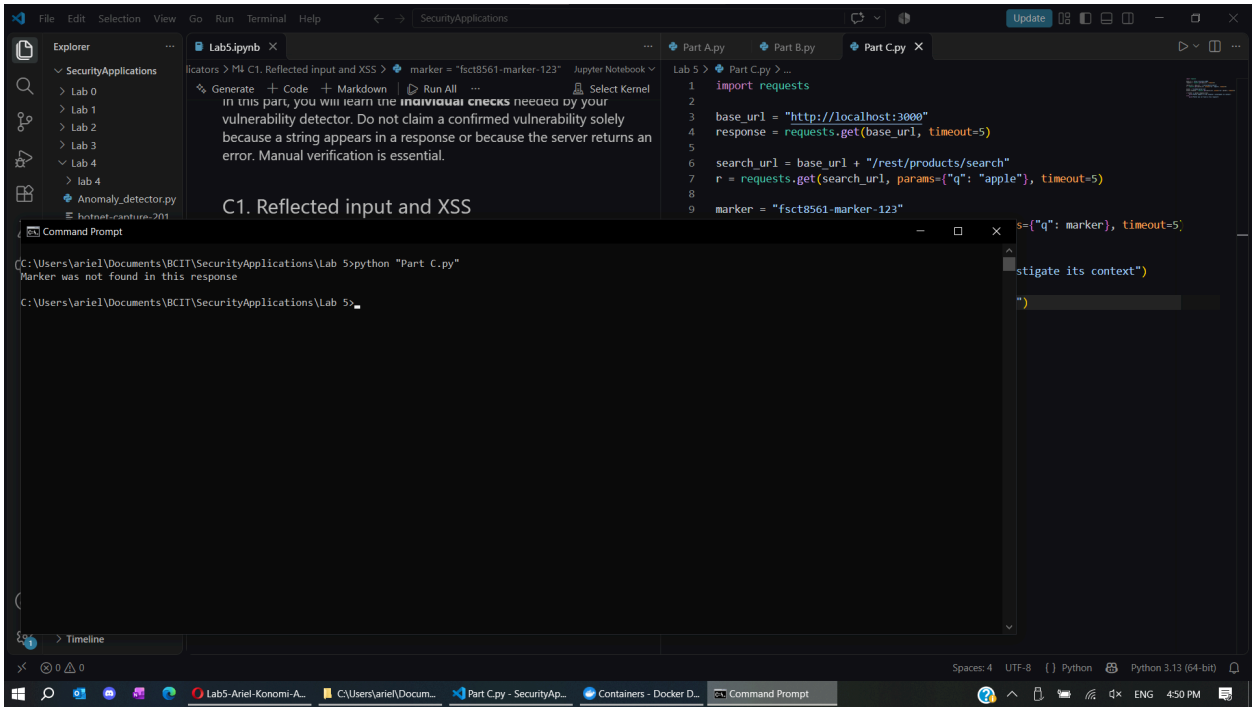
Practice G:

Explain why a scanner should include a timeout and handle `requests.RequestException`.

A scanner should include a timeout to prevent an indefinite wait time. It should also handle requests.RequestException to catch network errors gracefully and without crashing

Part C

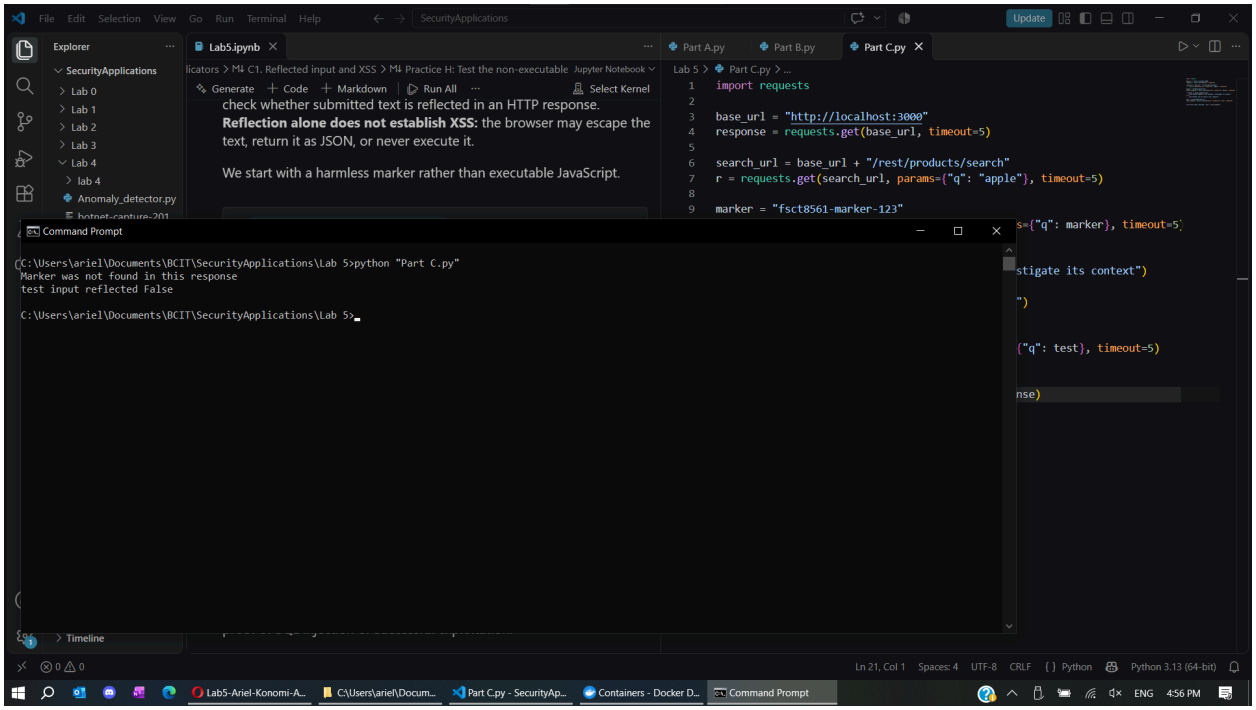
C1



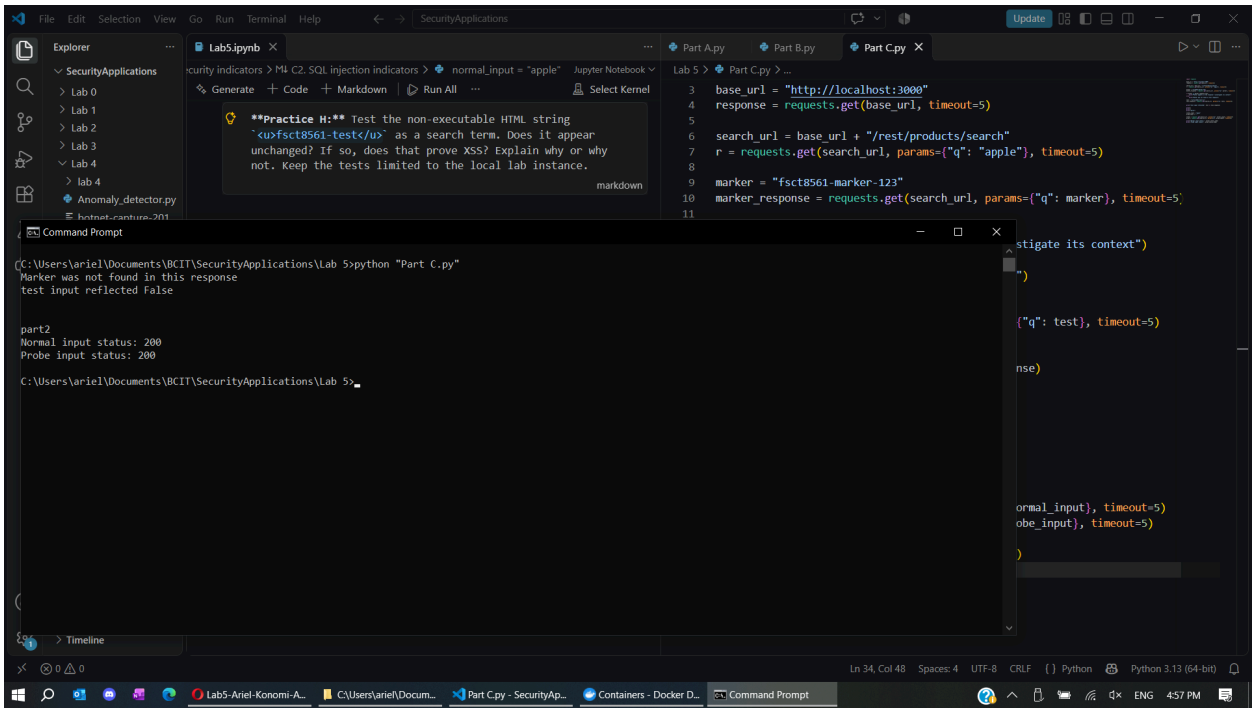
Practice H:

Test the non-executable HTML string `<u>fsct8561-test</u>` as a search term. Does it appear unchanged? If so, does that prove XSS? Explain why or why not. Keep the tests limited to the local lab instance.

The HTML string does appear to not change, however that does not prove XSS.


Lab5.ipynb
Part A.py
Part B.py
Part C.py
check whether submitted text is reflected in an HTTP response.
Reflection alone does not establish XSS: the browser may escape the text, return it as JSON, or never execute it.
We start with a harmless marker rather than executable JavaScript.
Command Prompt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>python "Part C.py"
Marker was not found in this response
test input reflected False
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>

C2

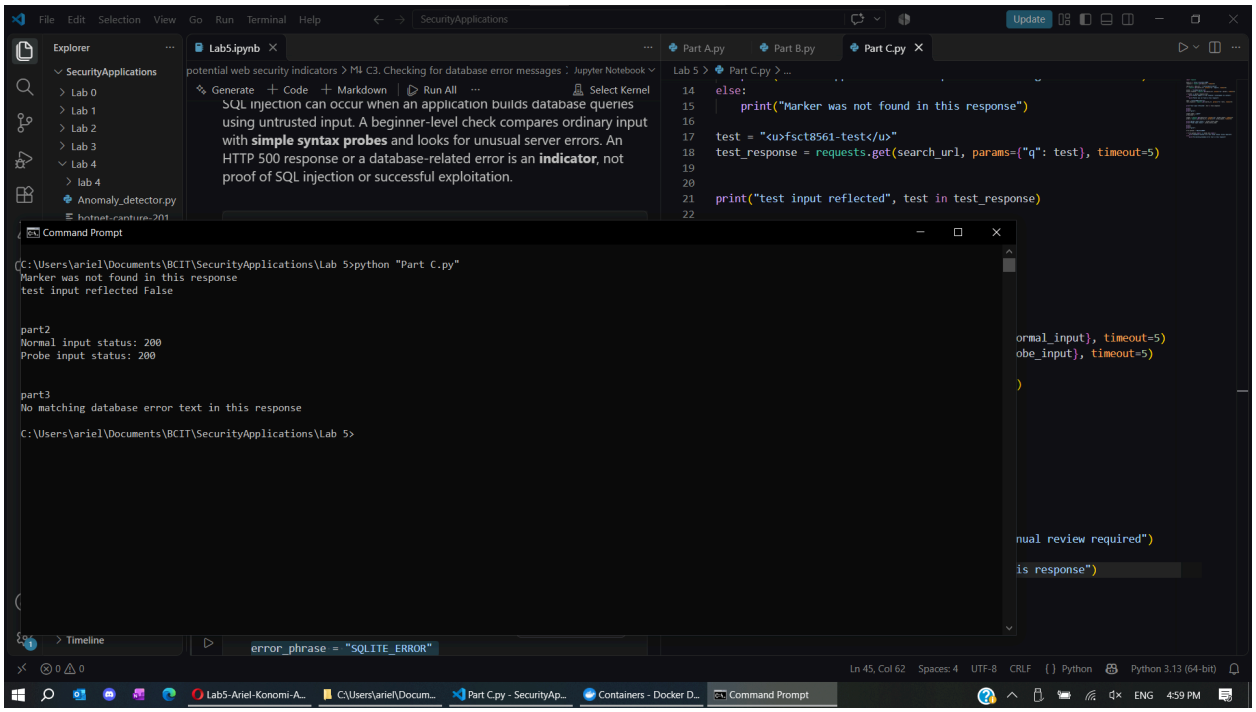

Lab5.ipynb
Part A.py
Part B.py
Part C.py
Practice H: Test the non-executable HTML string `<u>fsct8561-test</u>` as a search term. Does it appear unchanged? If so, does that prove XSS? Explain why or why not. Keep the tests limited to the local lab instance.
markdown
Command Prompt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>python "Part C.py"
Marker was not found in this response
test input reflected False
part2
Normal input status: 200
Probe input status: 200
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>

Practice I:

Explain what you observed. Why is comparing the probe response to a normal baseline more informative than looking at the probe alone?

A normal input like apple or banana shows and gives a baseline for how the response is supposed to look like. By looking at the baseline, you can more easily understand what the probe responses

C3



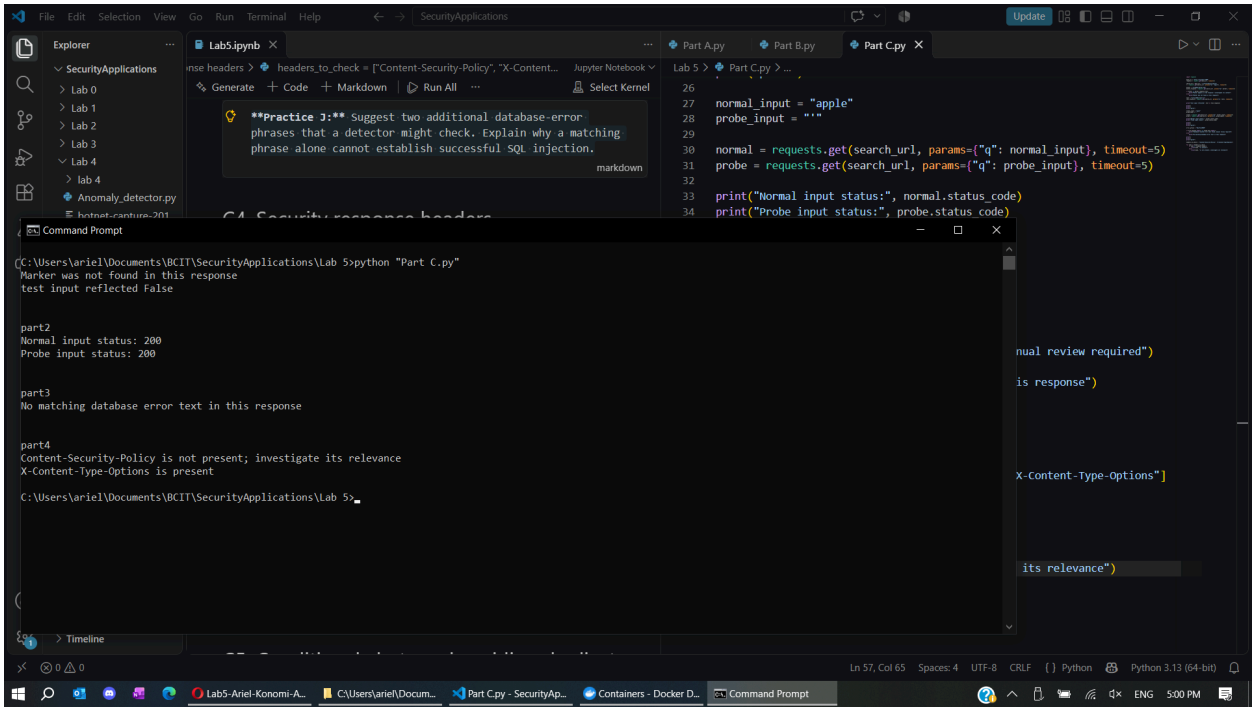
Practice J:

Suggest two additional database-error phrases that a detector might check. Explain why a matching phrase alone cannot establish successful SQL injection.

1. Syntax error
2. Database error

A matching phrase cannot establish successful SQL injection as it can only indicate abnormal database handling

C4

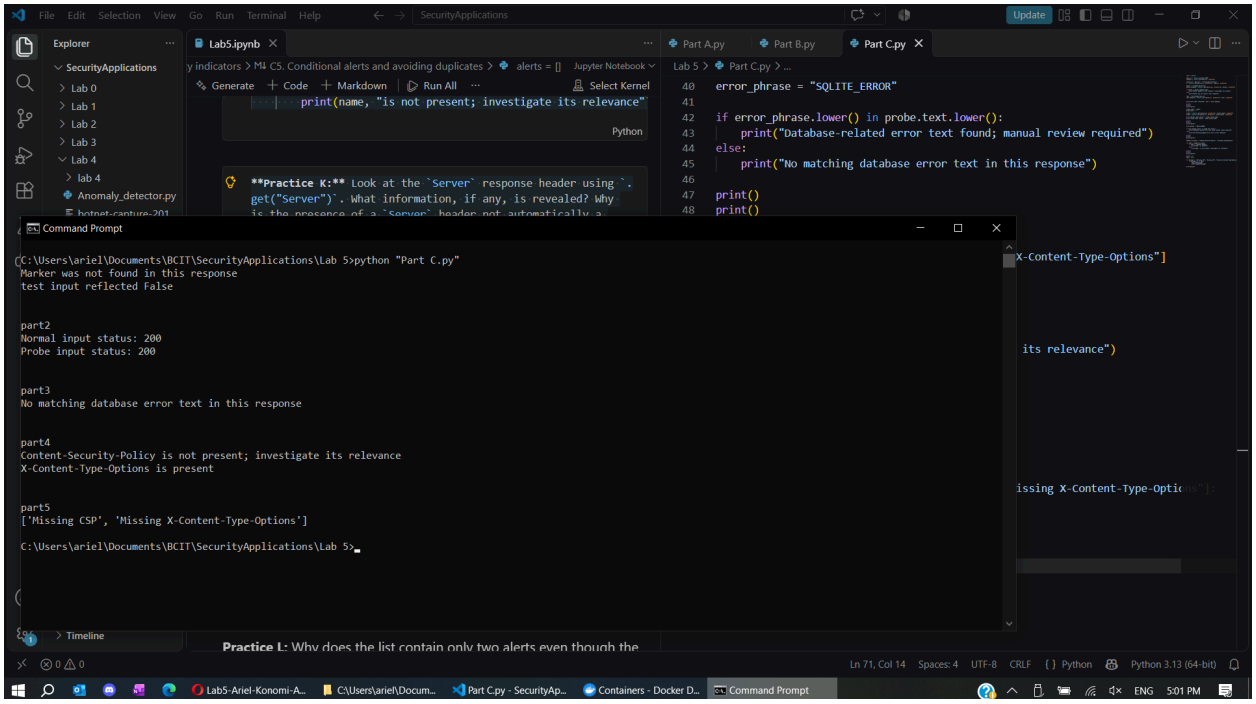


Practice K:

Look at the `Server` response header using `.get("Server")`. What information, if any, is revealed? Why is the presence of a `Server` header not automatically a critical vulnerability?

The content security policy header is not present, while the x content type options is.
The server header presence is not automatically a critical vulnerability. It does however reveal some software or framework details.

C5

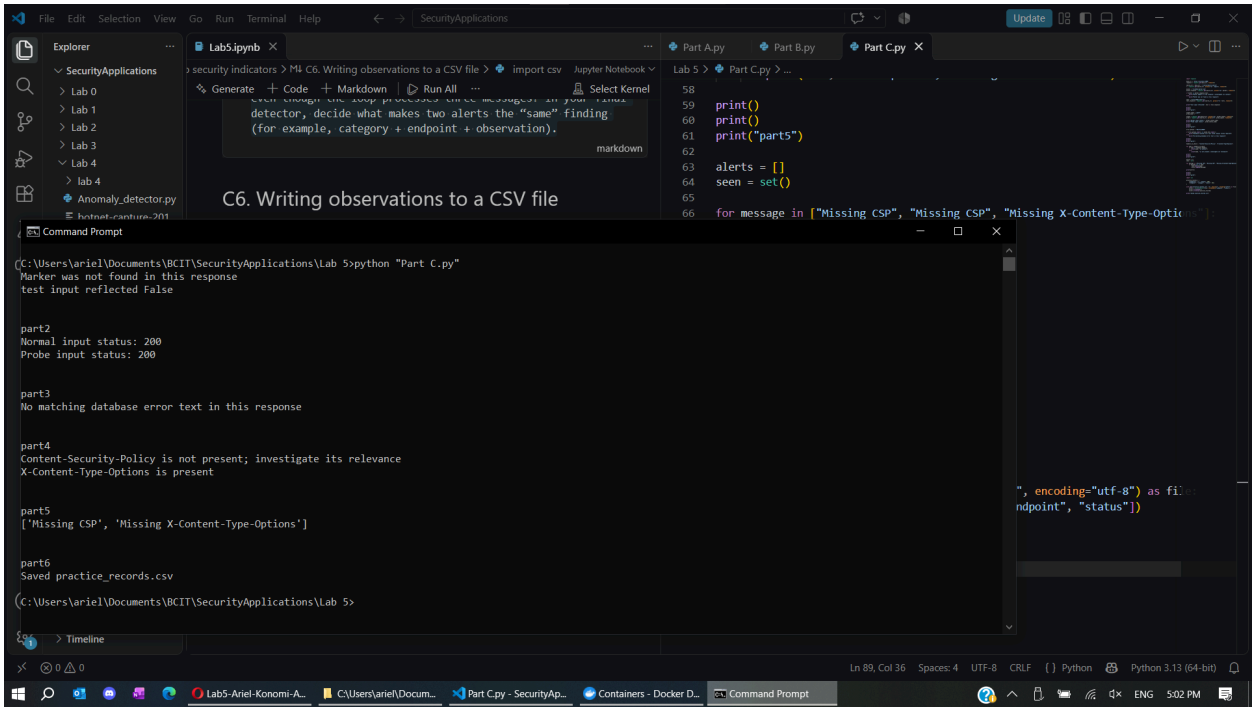


Practice L:

Why does the list contain only two alerts even though the loop processes three messages? In your final detector, decide what makes two alerts the “same” finding (for example, category + endpoint + observation).

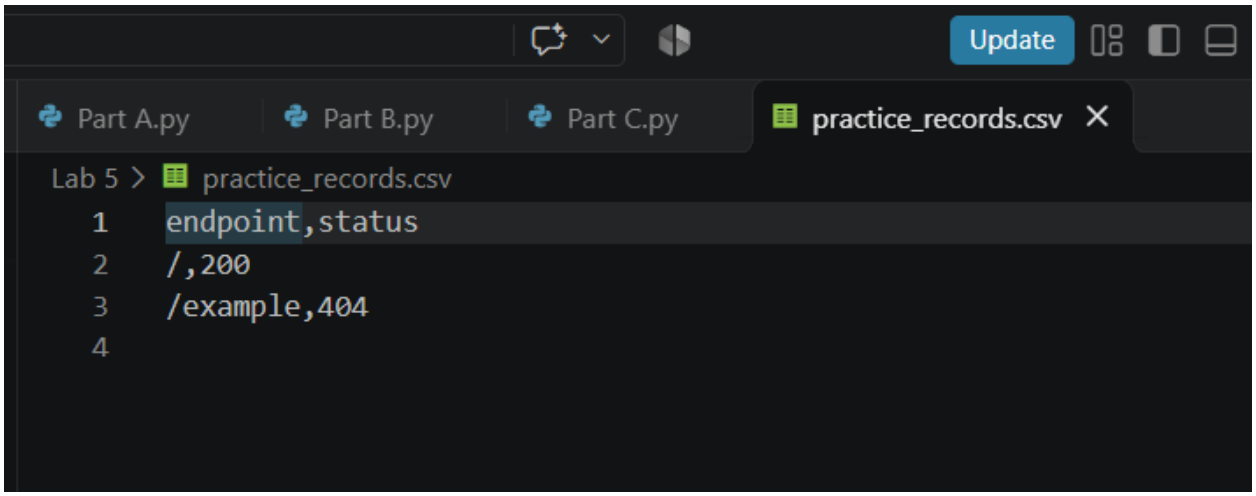
The list only contains two alerts due to duplicate messages. If the message is a duplicate, it automatically gets filtered

C6



Practice M:

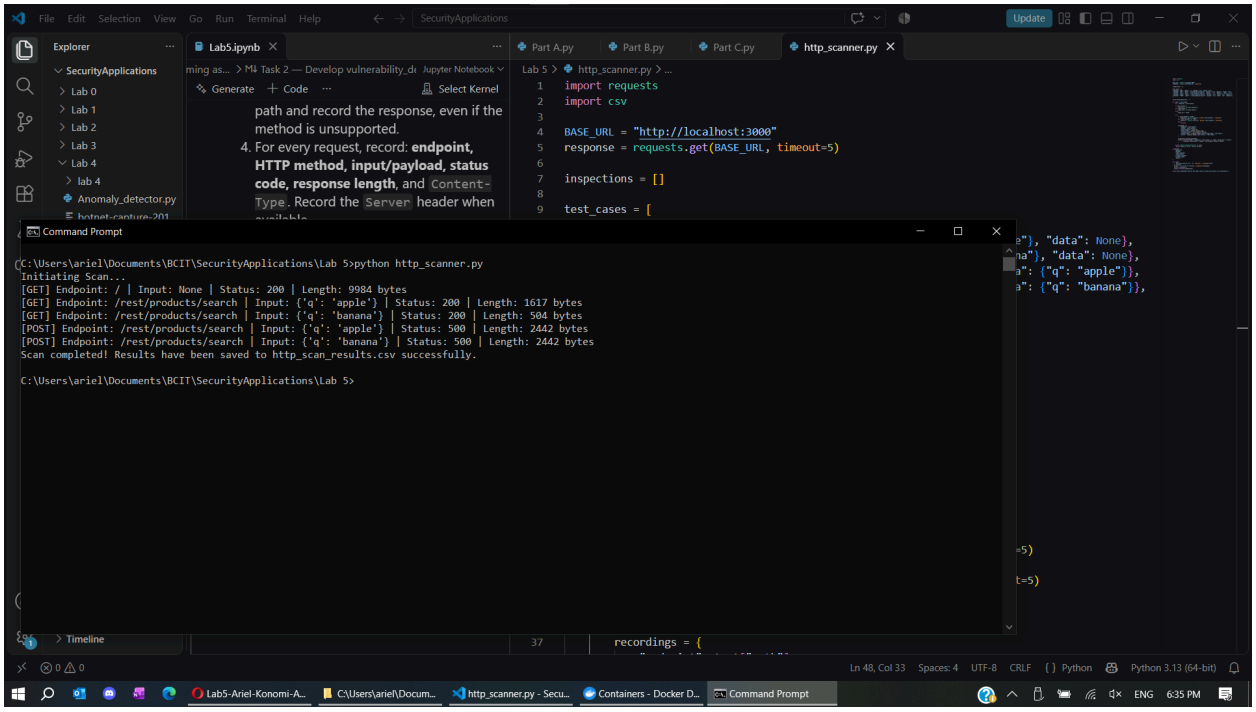
Open `practice_records.csv` and identify its column headings and two rows. Why is CSV useful for the final lab report?

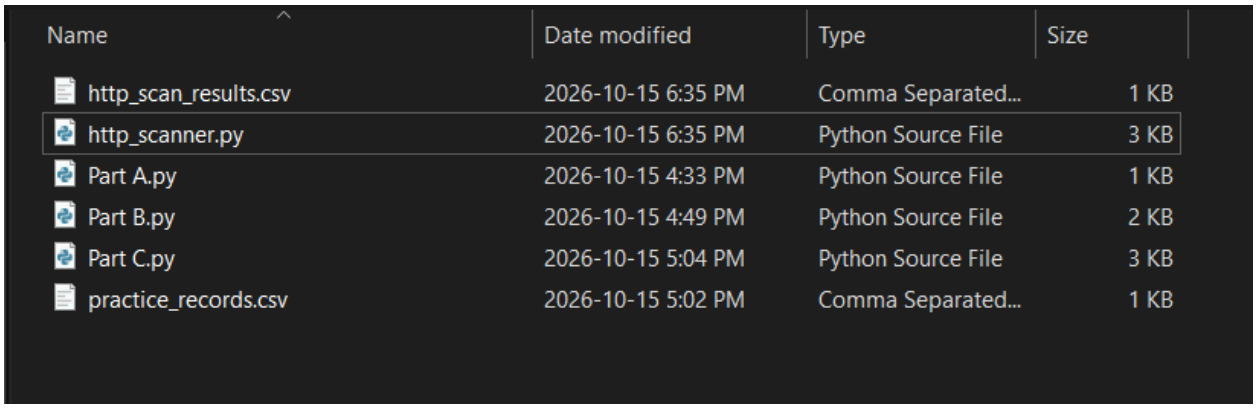


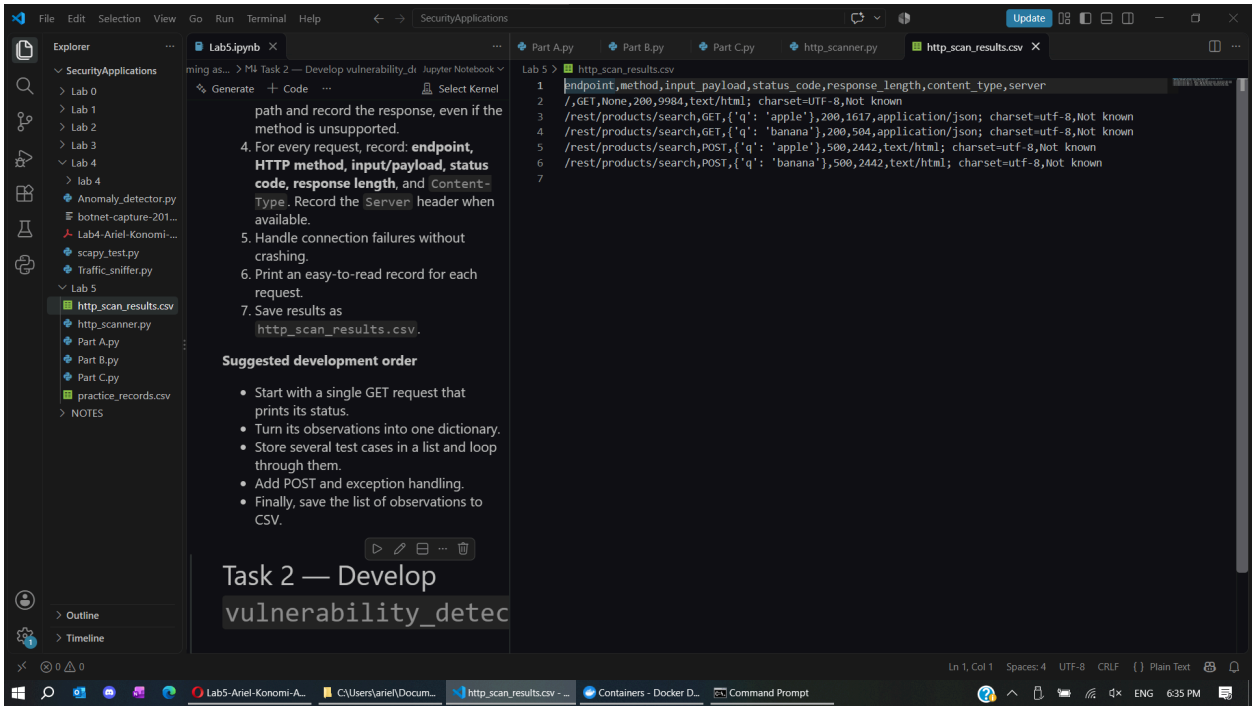
CSV is useful for the final report as it provides a portable format of the data retrieved that can easily be read and understood

Part D

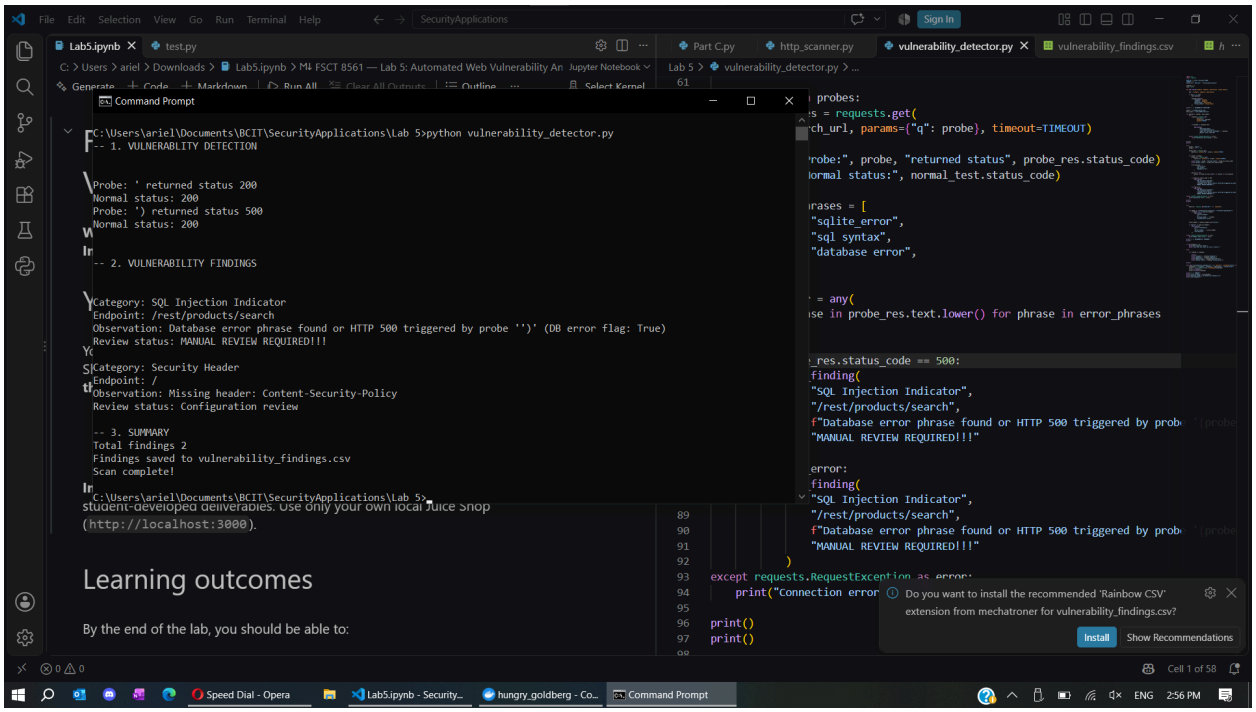
Task 1

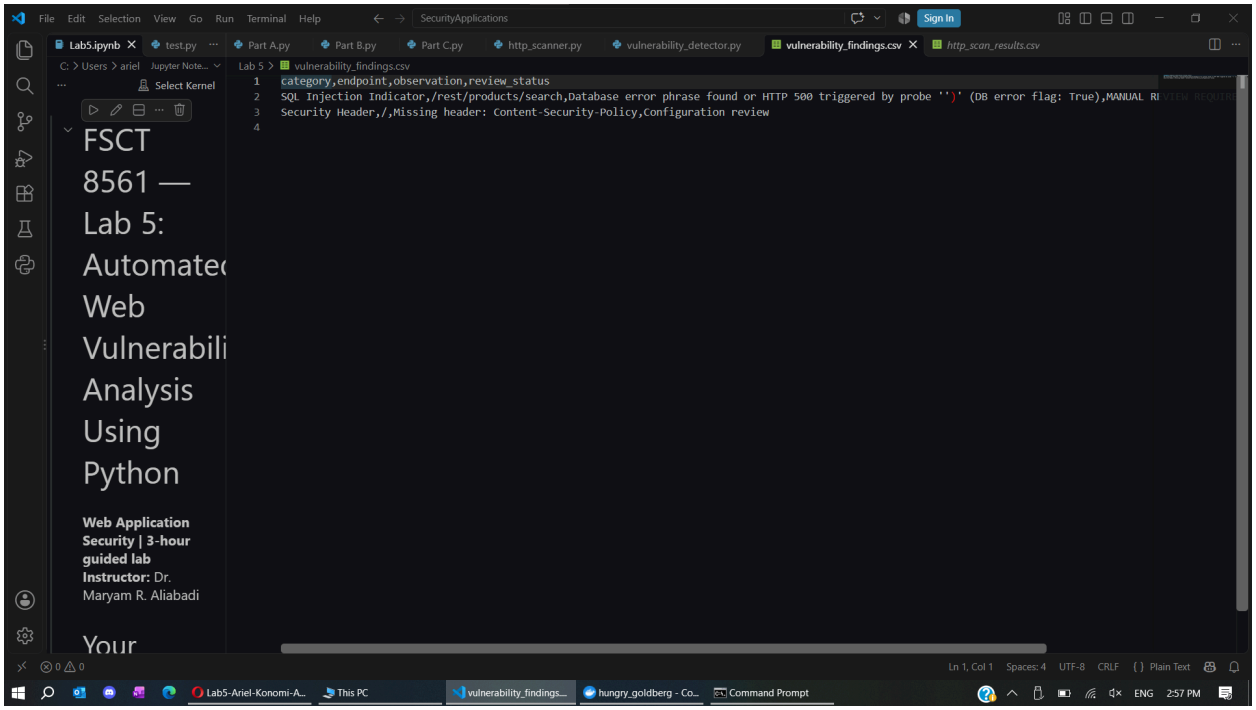

Command Prompt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>python http_scanner.py
Initiating Scan...
[GET] Endpoint: / | Input: None | Status: 200 | Length: 9984 bytes
[GET] Endpoint: /rest/products/search | Input: {'q': 'apple'} | Status: 200 | Length: 1617 bytes
[GET] Endpoint: /rest/products/search | Input: {'q': 'banana'} | Status: 200 | Length: 504 bytes
[POST] Endpoint: /rest/products/search | Input: {'q': 'apple'} | Status: 500 | Length: 2442 bytes
[POST] Endpoint: /rest/products/search | Input: {'q': 'banana'} | Status: 500 | Length: 2442 bytes
Scan completed! Results have been saved to http_scan_results.csv successfully.
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>


Name
Date modified
Type
Size
http_scan_results.csv 2026-10-15 6:35 PM Comma Separated... 1 KB
http_scanner.py 2026-10-15 6:35 PM Python Source File 3 KB
Part A.py 2026-10-15 4:33 PM Python Source File 1 KB
Part B.py 2026-10-15 4:49 PM Python Source File 2 KB
Part C.py 2026-10-15 5:04 PM Python Source File 3 KB
practice_records.csv 2026-10-15 5:02 PM Comma Separated... 1 KB


path and record the response, even if the method is unsupported.
4. For every request, record: **endpoint, HTTP method, input/payload, status code, response length**, and `Content-Type`. Record the `Server` header when available.
5. Handle connection failures without crashing.
6. Print an easy-to-read record for each request.
7. Save results as `http_scan_results.csv`.
Suggested development order
- Start with a single GET request that prints its status.
- Turn its observations into one dictionary.
- Store several test cases in a list and loop through them.
- Add POST and exception handling.
- Finally, save the list of observations to CSV.
Task 2 — Develop vulnerability_detec
endpoint,method,input_payload,status_code,response_length,content_type,server
/,GET,None,200,9984,text/html; charset=UTF-8,Not known
/rest/products/search,GET,{'q': 'apple'},200,1617,application/json; charset=utf-8,Not known
/rest/products/search,GET,{'q': 'banana'},200,504,application/json; charset=utf-8,Not known
/rest/products/search,POST,{'q': 'apple'},500,2442,text/html; charset=utf-8,Not known
/rest/products/search,POST,{'q': 'banana'},500,2442,text/html; charset=utf-8,Not known

Task 2


Command Prompt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>python vulnerability_detector.py
-- 1. VULNERABLITY DETECTION
Probe: ' returned status 200
Normal status: 200
Probe: ') returned status 500
Normal status: 200
-- 2. VULNERABILITY FINDINGS
Category: SQL Injection Indicator
Endpoint: /rest/products/search
Observation: Database error phrase found or HTTP 500 triggered by probe '')' (DB error flag: True)
Review status: MANUAL REVIEW REQUIRED!!!
Category: Security Header
Endpoint: /
Observation: Missing header: Content-Security-Policy
Review status: Configuration review
-- 3. SUMMARY
Total findings 2
Findings saved to vulnerability_findings.csv
Scan complete!
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 5>
(http://localhost:3000).
Learning outcomes
By the end of the lab, you should be able to:
Do you want to install the recommended 'Rainbow CSV' extension from mechatroner for vulnerability_findings.csv?
Install
Show Recommendations



Task 3

For this lab, a local OWASP juice shop application was set up at http://localhost:3000. A scanner and detector were built to evaluate the server response headers and behaviour. The tests were done over HTTP, not HTTPS. During some of the results, HTTP 500 popped up. This means that they would need to be checked manually. One of the first things that was noticed was that the content security policy (CSP) header was missing. Due to it being an important header, helping mitigate XSS risks, the gap would need to be checked. The reason it was missing however could be due to the fact that this was a local environment test. X content type options was detected during this test. The server header also appeared to be empty/missing, but this does not automatically mean that the configuration was incorrect.

Task 5

Multiple tests were done for this lab. The first tests were the basic searches for the words apple and banana. These GET requests both gave back a HTTP 200 status. Another test was to try and send a POST request, but that immediately came back with a HTTP 500 status. A test with a SQL syntax probe was made "')". This one also gave a HTTP 500 error status. Due to the way the script was built, I was able to get some database related strings in the response as well. This specific error says that a potential SQL injection might have happened but a manual check of the logs would be needed to fully verify.

If I was the security lead for the Juice shop, I would investigate the errors created from the server. I would also investigate the missing headers from the server. All of this would need to be done manually of course to fully read and analyze what is happening on the backend. Lab 5 builds upon lab 4 by moving from PCAP to active application layer testing. To be able to do this, the request library is used, and packet sniffers are used to analyze network frames. The application uses HTTP instead of HTTPS. This is done as if HTTPS was used, a network packet capturer would only be able to see encrypted text. Automated findings should always be checked manually as findings often are generated and false positives are given as a result.